MODULE 2

Prompting Techniques: Fundamentals

Duration	Format	Audience	Day / Position
45 min	**Lecture + Demo**	**Mixed Engineers**	**Day 1 / Second**

LEARNING OBJECTIVES

By the end of this module, participants will be able to:

1. Distinguish between system prompts and task prompts in Claude Code and explain when to use each
2. Apply zero-shot, one-shot, and few-shot prompting patterns to generate code with predictable quality
3. Use Chain-of-Thought (CoT) reasoning to guide Claude Code through complex, multi-step coding tasks
4. Practice prompt hygiene: effective context packing, @-file references, snippet safety, and token management
5. Complete a live prompting exercise solving a TI-relevant Python scripting task using the techniques learned

SESSION TIMING BREAKDOWN

0:00–0:08	**Section 1: The Language of AI — Why Prompting Matters**
0:08–0:18	**Section 2: System Prompts vs. Task Prompts**
0:18–0:28	**Section 3: Zero-Shot, One-Shot & Few-Shot Patterns**
0:28–0:38	**Section 4: Chain-of-Thought Reasoning**
0:38–0:43	**Section 5: Prompt Hygiene — Context, References & Safety**
0:43–0:45	**Wrap-Up: Live Demo & Transition to Module 3**

0:00 – 0:08	SECTION 1: The Language of AI — Why Prompting Matters

1. The Language of AI — Why Prompting Matters

1.1 What Is a Prompt?

A prompt is the natural language instruction you give to an AI model. In Claude Code, every command you type in the terminal is a prompt. The quality of your prompt directly determines the quality of the code Claude generates.

> *"Prompt engineering is to AI-assisted coding what requirements gathering is to software engineering. Garbage in, garbage out. Precise in, precise out."*

Think of it this way: in traditional coding, you translate requirements into code. In AI-assisted coding, you translate requirements into prompts, and the AI translates those into code. Your prompting skill is now the critical bottleneck in your productivity.

1.2 The Prompt → Response Loop

Every interaction with Claude Code follows this cycle:

Step	What Happens	Your Responsibility
1. Context	Claude reads your codebase, CLAUDE.md, and conversation history	Ensure Claude has the right context (not too much, not too little)
2. Prompt	You type a natural language instruction	Be specific, structured, and include constraints
3. Processing	Claude plans an approach and generates code/commands	Understand that Claude may interpret ambiguity differently than you expect
4. Output	Claude presents code, diffs, or executes commands	Review every output. Accept, reject, or refine.
5. Iteration	You refine: correct mistakes, add detail, redirect	Use conversation history to iteratively improve results

> ☆ **KEY POINT:** Prompting is iterative, not one-shot. The best results come from starting with a clear prompt, reviewing the output, and refining through 2–3 follow-up prompts. Don't expect perfection on the first try.

1.3 The Anatomy of an Effective Prompt

Every effective prompt for Claude Code contains some or all of these elements:

Element	Example
Role / Context	“You’re helping me build a Python data processing script for sensor data...”
Specific Task	“Create a function that reads CSV files from a directory and merges them...”
Constraints	“Use pandas, handle missing values with forward-fill, add type hints...”
Output Format	“Return a DataFrame. Add a docstring in Google style. Include error handling.”
Examples (optional)	“Input: ['sensor_01.csv', 'sensor_02.csv'] → Output: merged DataFrame with timestamp index”
Negative Constraints	“Do NOT use global variables. Do NOT modify the original CSV files.”

0:08 – 0:18	SECTION 2: System Prompts vs. Task Prompts

2. System Prompts vs. Task Prompts

In Claude Code, there are two fundamentally different types of prompts that work together. Understanding the distinction is essential for consistent, high-quality results.

2.1 System Prompts: Persistent Context

A system prompt sets the "personality," constraints, and global behavior for Claude. In Claude Code, the primary system prompt mechanism is the CLAUDE.md file:

System Prompt Source	How It Works
CLAUDE.md (project root)	Automatically loaded every time Claude Code starts in that directory. Defines project standards, coding conventions, and safety guardrails.
CLAUDE.md (nested dirs)	Additional context for subdirectories. Claude Code reads all CLAUDE.md files from root down to the current directory.
~/.claude/CLAUDE.md	User-level defaults that apply to ALL your projects. Great for personal style preferences.
--append-system-prompt	CLI flag to add instructions for a specific session. Useful for one-off behavior changes.
--system-prompt	CLI flag to completely replace the default system prompt. Advanced use only — removes Claude Code's built-in capabilities.

Here's an example CLAUDE.md that acts as a system prompt for a TI Python project:

```
# CLAUDE.md

## Project
TI Sensor Data Pipeline - Python 3.11 on Ubuntu 22.04

## Standards
- Type hints on ALL function signatures
- Google-style docstrings on ALL public functions
- Max function length: 25 lines
- Use pathlib instead of os.path
- Use logging module, never print() for debugging

## Testing
- pytest with --strict-markers
- Naming: test_{function_name}_{scenario}_{expected}
```

```
- Minimum coverage: 85%

## Safety
- NEVER hardcode credentials or file paths
- Use environment variables for all configuration
- Validate all input data before processing
```

☆ **KEY POINT:** CLAUDE.md is the most powerful prompting tool in Claude Code. It's applied automatically to every interaction, ensuring consistency across your entire team. Think of it as a persistent system prompt that lives in your repository and gets version-controlled alongside your code.

2.2 Task Prompts: Single Interactions

A task prompt is what you type in the Claude Code interactive session or pass via claude -p. It's a specific request for a specific action:

Task Prompt Type	Example	When to Use
Direct instruction	"Create a function that calculates moving averages"	Simple, well-defined tasks with clear output
Question	"What does the process_batch() function do?"	Understanding code, exploring architecture
Modification	"Add error handling to the read_sensor_data function"	Refactoring, improving existing code
Multi-step	"Read the CSV, clean missing values, calculate stats, and write results to a new file"	Sequential workflows (better to decompose)
Debugging	"Here's the error: [traceback]. Fix it."	Paste errors directly for diagnosis
Review	"Review this function for edge cases and security issues"	Code quality checks, pre-commit review

2.3 How System and Task Prompts Work Together

The system prompt (CLAUDE.md) sets the rules; the task prompt provides the specific request. Claude Code combines both:

```
# Your CLAUDE.md says:
#   - Use type hints on all functions
#   - Google-style docstrings
#   - Max 25 lines per function

# Your task prompt:
> Create a function to validate sensor readings
```

```
# Claude Code automatically applies CLAUDE.md standards:
# - Adds type hints (from CLAUDE.md)
# - Adds Google docstring (from CLAUDE.md)
# - Keeps function under 25 lines (from CLAUDE.md)
# - Implements validation logic (from your prompt)
```

QUICK CHECK (1 minute)

- Think about your current project at TI. What are 3 rules you'd put in a CLAUDE.md file?
- Examples: coding standards, naming conventions, file structure rules, testing requirements, forbidden patterns
- We'll revisit this in Module 4 when you create your own CLAUDE.md

0:18 – 0:28	SECTION 3: Zero-Shot, One-Shot & Few-Shot Patterns

3. Zero-Shot, One-Shot & Few-Shot Patterns

These are the foundational prompting patterns used across all AI systems. They describe how many examples you provide to Claude before making your request.

3.1 Zero-Shot Prompting

Give Claude a direct instruction with no examples. Claude relies entirely on its training to understand what you want.

Example — Zero-Shot:

```
> Create a Python function that validates email addresses
> using regex. Return True for valid, False for invalid.
> Add type hints and a docstring.
```

When to Use Zero-Shot	When to Avoid Zero-Shot
Standard coding tasks	Tasks requiring specific output format
Well-known patterns	Domain-specific conventions Claude may not know
Simple function creation	When you need exact naming or structure
Quick prototyping	Complex multi-step transformations

3.2 One-Shot Prompting

Provide one example of the desired input/output pattern before your actual request. This dramatically improves consistency.

Example — One-Shot:

```
> I need functions that follow this pattern:
>
> def validate_temperature(value: float) -> tuple[bool, str]:
>     """Validate temperature reading from sensor.
>
>     Args:
>         value: Temperature in Celsius.
>
>     Returns:
>         Tuple of (is_valid, message).
>     """
>     if -40 <= value <= 150:
```

```
>         return True, "OK"
>     return False, f"Temperature {value}C out of range [-40, 150]"
>
> Now create similar validation functions for:
> 1. Voltage (0-5V range)
> 2. Pressure (0-1000 kPa range)
> 3. Humidity (0-100% range)
```

> ☆ **KEY POINT:** One-shot prompting is the sweet spot for most Claude Code tasks. A single example communicates your preferred function structure, naming convention, docstring style, and error handling pattern more precisely than paragraphs of description.

3.3 Few-Shot Prompting

Provide 2–5 examples to establish a clear pattern. Especially useful when the transformation is complex or the output format is non-obvious.

Example — Few-Shot:

```
> Convert these TI error codes to human-readable messages.
> Follow the exact pattern shown:
>
> Input: "ERR_TEMP_HIGH_001" -> Output: {
>   "code": "ERR_TEMP_HIGH_001",
>   "severity": "critical",
>   "message": "Temperature exceeded upper limit",
>   "action": "Immediate shutdown required"
> }
>
> Input: "WARN_VOLT_LOW_003" -> Output: {
>   "code": "WARN_VOLT_LOW_003",
>   "severity": "warning",
>   "message": "Voltage below nominal range",
>   "action": "Schedule maintenance check"
> }
>
> Now convert these error codes:
> ERR_PRESS_SPIKE_007, WARN_HUM_HIGH_012, ERR_FLOW_STOP_015
```

3.4 Choosing the Right Pattern

Pattern	# Examples	Best For
Zero-Shot	0	Standard tasks, prototyping, simple functions, well-known patterns
One-Shot	1	Establishing function structure, naming conventions, output format

Few-Shot	2–5	Complex transformations, domain-specific formats, data mapping patterns

⚠ **WARNING:** More examples is not always better. Beyond 5 examples, you risk consuming valuable context window tokens without improving output quality. If you need more than 5 examples to communicate your intent, consider creating a CLAUDE.md section or a reference file and using @-references instead.

0:28 – 0:38	SECTION 4: Chain-of-Thought Reasoning

4. Chain-of-Thought Reasoning

Chain-of-Thought (CoT) prompting asks Claude to reason step-by-step before generating code. This is the single most effective technique for complex coding tasks that require planning, multi-step logic, or architectural decisions.

4.1 What Is Chain-of-Thought?

Instead of asking Claude to jump directly to code, you ask it to think through the problem first. This mirrors how experienced engineers approach complex tasks: plan before you code.

Without CoT (Direct)	With CoT (Step-by-Step)
"Create a function to process sensor data"	"First, outline the steps needed to process sensor data: reading, validation, cleaning, transformation, and output. Then implement each step as a separate function."
Claude jumps to code immediately	Claude plans the approach, then codes with clear structure
May miss edge cases	More likely to handle edge cases during planning phase
Often produces monolithic functions	Produces well-decomposed, testable functions

4.2 CoT Prompting Techniques for Claude Code

Technique 1: Explicit Step-by-Step

```
> I need a Python module for processing sensor batch files.
> Before writing code, please:
> 1. List the steps needed for the complete workflow
> 2. Identify the functions you'll need
> 3. Note any edge cases or error conditions
> 4. Then implement the solution with tests
```

Technique 2: "Think Before You Code"

```
> Think step by step about how to implement a data pipeline
> that reads CSV files, validates each row against a schema,
> handles missing values, and outputs a clean parquet file.
> Explain your approach first, then write the code.
```

Technique 3: Plan Mode

Claude Code has a built-in plan mode that enforces CoT behavior:

```
# Activate plan mode via slash command:
/plan

# Or toggle via shift+tab in interactive mode
# In plan mode, Claude ONLY creates plans, never executes code.
# This is perfect for complex tasks where you want to review
# the approach before any files are changed.

# Example workflow:
# 1. Enter plan mode (/plan)
# 2. Describe your task
# 3. Claude creates a detailed plan
# 4. You review and approve/modify the plan
# 5. Exit plan mode and ask Claude to execute the plan
```

☆ **KEY POINT:** Plan mode is your safety net for complex changes. Use it whenever a task touches more than 2–3 files, requires architectural decisions, or could introduce breaking changes. Review the plan, then let Claude execute.

4.3 Extended Thinking

Claude Code supports extended thinking, where Claude takes more time to reason deeply before responding. This is especially useful for:

Use Case	Why Extended Thinking Helps
Complex algorithms	More time to consider edge cases and optimize
Architecture decisions	Better evaluation of trade-offs between approaches
Debugging tricky issues	Deeper root cause analysis before suggesting fixes
Multi-file refactoring	More careful planning of cross-file dependencies
Test strategy design	More thorough coverage of test scenarios

```
# Enable extended thinking in config:
/config
# Navigate to thinking mode and enable it

# Or set via CLI:
claude --thinking-budget high

# Extended thinking uses more tokens but produces
# higher-quality reasoning for complex tasks.
```

4.4 When NOT to Use CoT

CoT is not always necessary. For simple tasks, it adds overhead without benefit:

- Simple function creation with clear specifications — use zero-shot
- File renaming, import updates, or mechanical refactoring — direct instruction
- Running existing tests or checking status — just ask
- Generating docstrings or type hints for existing code — direct instruction

☆ **KEY POINT:** Use CoT for tasks where you would need to think before coding if you were doing it manually. If the task is mechanical or well-defined, skip CoT and use direct prompting for speed.

0:38 – 0:43	SECTION 5: Prompt Hygiene — Context, References & Safety

5. Prompt Hygiene: Context, References & Safety

Even with great prompting patterns, poor context management will degrade results. This section covers the practical skills that separate effective Claude Code users from frustrated ones.

5.1 Context Packing with @-References

Claude Code's @-reference system lets you precisely control what files and directories Claude sees:

@-Reference	What It Does
@filename.py	Include a specific file in Claude's context
@src/core/	Include all files in a directory
@tests/unit/test_calc.py	Include a specific test file for reference
@requirements.txt	Show Claude your dependencies
@.github/workflows/ci.yml	Include CI config for Claude to understand your pipeline

Effective @-reference usage:

```
# BAD: Vague prompt with no context
> Fix the bug in the data processing code

# GOOD: Specific prompt with precise context
> Fix the TypeError in @src/core/processor.py line 45.
> The test @tests/unit/test_processor.py::test_empty_input
> is failing. Here's the traceback: [paste error]

# GOOD: Reference related files for refactoring
> Refactor @src/utils/validators.py to use the same
> patterns as @src/utils/formatters.py
```

5.2 Managing the Context Window

Claude Code has a limited context window. When conversations get long, context management becomes critical:

Tool	Command	When to Use

/compact	/compact	Summarize conversation history to free up context. Use when Claude starts forgetting earlier instructions.
/clear	/clear	Completely reset the conversation. Use when switching to a totally different task.
Focused @-refs	@specific_file.py	Only reference the files Claude needs. Don't dump entire directories.
/context	/context	View what's currently in Claude's context window. Check if it's overloaded.
New session	exit + claude	Start fresh when context is polluted with irrelevant history.

☆ **KEY POINT:** The /compact command is your best friend for long sessions. It tells Claude to summarize the conversation so far, freeing up context window space while preserving the key decisions and context from earlier in the session.

5.3 Snippet Safety: What to Include in Prompts

When pasting code into prompts, follow these rules:

✓ SNIPPET SAFETY RULES

- DO paste error messages and tracebacks verbatim — Claude needs the exact error to diagnose
- DO include relevant function signatures and class definitions for context
- DO include test output so Claude can see what's passing and failing
- DO NOT paste entire files when only a specific function is relevant — use @-references instead
- DO NOT include credentials, API keys, or secrets (even in comments or config examples)
- DO NOT paste large datasets inline — reference the file with @ or describe the schema
- SANITIZE proprietary names before pasting: use generic names like process_data instead of ti_fab7_yield_calc

5.4 Common Prompt Anti-Patterns

Anti-Pattern	Better Alternative
"Fix the code" (no context)	"Fix the TypeError in @processor.py line 45. Error: [paste traceback]"
"Make it better" (vague)	"Refactor to reduce cyclomatic complexity. Extract helper functions for readability."
"Write tests" (underspecified)	"Write pytest tests for @calc.py covering: valid input, empty input, None input, and overflow."
"Do everything" (too broad)	"Start with step 1: create the data validation module. We'll add processing next."

Pasting 500 lines inline	"See @src/core/engine.py. Focus on the transform_data method (lines 120–180)."
Repeating failed prompt	"That didn’t work because [reason]. Try a different approach: [new direction]."

0:43 – 0:45	WRAP-UP: Live Demo & Transition to Module 3

6. Live Demo: TI Python Scripting Task

Let's put it all together. Here's a real-world prompting workflow that combines every technique from this module:

6.1 The Task

Create a Python module that reads sensor log files (CSV), validates the data, calculates summary statistics, and generates a report.

6.2 The Prompting Workflow

🚀 **LIVE DEMO: Full Prompting Workflow**

- Step 1 (System Prompt): Show the CLAUDE.md is already loaded with project standards
- Step 2 (Plan Mode): Enter /plan and describe the full task. Review Claude's plan.
- Step 3 (CoT): "Think step by step about what functions we need for this module. List them first."
- Step 4 (One-Shot): Provide one example function to establish the pattern, then ask Claude to generate the rest
- Step 5 (Zero-Shot): Ask Claude to generate pytest tests for the module
- Step 6 (Iteration): Review the output, identify an issue, refine with a follow-up prompt
- Step 7 (Context): Use @-references to show Claude the test results and ask it to fix failures

---

MODULE 2 RECAP

✅	Prompting is a skill, not just typing. Specific, structured prompts produce dramatically better code than vague ones.
✅	System prompts (CLAUDE.md) set persistent rules; task prompts are your specific requests. They work together.
✅	Zero-shot for simple tasks, one-shot for consistency, few-shot for complex patterns. One-shot is the sweet spot.
✅	Chain-of-Thought and Plan Mode force Claude to think before coding — essential for complex, multi-file tasks.
✅	Prompt hygiene matters: use @-references for context, /compact for long sessions, and sanitize all snippets.

✅	Common anti-patterns (vague prompts, no context, too broad) waste time. Be specific, constrained, and iterative.

QUICK REFERENCE CARD

Keep this reference handy during labs and exercises:

Technique	When to Use	Key Phrase
Zero-Shot	Simple, well-defined tasks	"Create a function that..."
One-Shot	Establishing patterns and conventions	"Follow this pattern: [example]. Now create..."
Few-Shot	Complex transformations, data mapping	"Here are 3 examples: [examples]. Now handle..."
CoT / Plan Mode	Multi-step, architectural, complex	"Think step by step..." or /plan
@-References	Providing file context	"See @filename.py, focus on..."
/compact	Long sessions, context overflow	Run /compact when Claude forgets things
Negative Constraints	Preventing unwanted patterns	"Do NOT use... Do NOT modify..."

TRANSITION TO MODULE 3

You’ve now mastered the fundamental prompting techniques. In Module 3, we’ll build on this foundation with advanced patterns: problem decomposition for multi-file changes, the ReAct framework, generation controls (temperature and extended thinking), iterative refinement strategies, and creating CLAUDE.md files for team-wide consistency.